Agent.py

```
from langchain_community.chat_models import ChatOllama
from langchain.agents import create_react_agent, AgentExecutor
from langchain.prompts import PromptTemplate

from agent.tools import get_tools
from agent.prompt import get_system_prompt

def create_agent():

    llm = ChatOllama(
        model="llama3:8b",
        temperature=0
    )

    tools = get_tools()

    # ✅ System prompt
    system_prompt = get_system_prompt()

    # ✅ Combined prompt
    prompt = PromptTemplate.from_template(f"""
{system_prompt}

---------------------------------------

You have access to the following tools:
{{tools}}

Available tool names:
{{tool_names}}

----------------------------------------

You MUST follow this format EXACTLY:

Question: {{input}}

Thought: <your reasoning>

Action: <tool name EXACTLY from [{{tool_names}}]>

Action Input: <input to the tool>
```

```
Observation: <tool result>

Final Answer: <your final response>

----------------------------------------

STRICT RULES:

- NEVER write sentences in Action
- Action must be EXACT tool name only

- Action Input MUST be:
  • ONLY valid SQL query (for SQL Tool)
  • ONLY None (for Prediction Tool)
  • NO explanations, NO comments, NO extra text

- Prediction Tool:
  • Requires NO input → always use: None
  • NEVER try to train model

- SQL Tool:
  • MUST receive ONLY executable SQL
  • NEVER include English text

- If model is not available:
  • Return the error
  • DO NOT retry
  • DO NOT switch tools

- ALWAYS use correct SQL Server syntax
- NEVER use LIMIT
- ALWAYS use TOP
- Use ONLY given schema

- Do NOT loop
- After one tool call → return Final Answer

----------------------------------------

Question: {{input}}
{{agent_scratchpad}}
""")

# ✅ Create agent
```

```
    agent = create_react_agent(
        llm=llm,
        tools=tools,
        prompt=prompt,
    )

    # ✅ Executor
    agent_executor = AgentExecutor(
        agent=agent,
        tools=tools,
        verbose=True,
        handle_parsing_errors=True,
        max_iterations=5,
        max_execution_time=60,
    )

    return agent_executor
```

---

prompt.py

```
def get_system_prompt():

    return """
You are an intelligent AI assistant working with a payment analytics system.

You have access to two tools:

1. SQL Tool → used for querying database
2. Prediction Tool → used for fraud and risk prediction

IMPORTANT:
- Prediction Tool is already trained
- NEVER attempt to train model
- NEVER generate training logic
--------------------------------------------------
DATABASE SCHEMA
--------------------------------------------------

Table: transactions
- id
- amount
- merchant_id
- date

Table: merchants
- id
```

- name
- country

IMPORTANT DATABASE RULES

- You are using SQL Server (NOT MySQL)
- ALWAYS use TOP (NEVER use LIMIT)

CRITICAL:
- transaction_count does NOT exist
- To get top merchants:
  → Use transactions table
  → Use COUNT(*) + GROUP BY merchant_id

CORRECT PATTERN:
```
SELECT TOP 5 merchant_id, COUNT(*) AS transaction_count
FROM transactions
GROUP BY merchant_id
ORDER BY transaction_count DESC;
```

- NEVER use COUNT(*) without GROUP BY
- NEVER ORDER BY COUNT(*) without aggregation
- DO NOT use merchants table for transaction counts

DECISION RULES

Use SQL Tool when:
- Query is about existing data
- Transactions, merchants, amounts

Use Prediction Tool when:
- Query is about fraud, risk, future prediction

SQL RULES

- Use only given schema
- Do NOT invent columns
- Use SQL Server syntax

- Use TOP for limiting results
- Default to TOP 10 if not specified
- Use WHERE, ORDER BY, GROUP BY properly

PREDICTION TOOL BEHAVIOR

- Prediction Tool automatically fetches required data from the database
- Do NOT ask user for input values
- Do NOT expect structured input from user
- Use Prediction Tool directly for risk/fraud queries
- The tool will internally:
  → fetch transaction data
  → run the machine learning model
  → return high-risk merchants

EXECUTION RULES (VERY IMPORTANT)

- Always use a tool when required
- ONLY ONE tool call allowed
- After Observation → immediately return Final Answer
- DO NOT loop
- DO NOT generate extra thoughts after result

FINAL ANSWER FORMAT

Final Answer:
<clear explanation of result>

Example:
"The top 5 merchants based on transaction count are..."

"""

Tools.py

```
from langchain.tools import Tool

from tools.sql_tool.sql_tool import sql_tool_func
from tools.prediction_tool.predictor_tool import prediction_tool_func
```

```
def get_tools():

    sql_tool = Tool(
        name="SQL Tool",
        func=sql_tool_func,
        description="""
Use ONLY for database queries.

- Input MUST be valid SQL
- Use for transactions, merchants, stored data
- Do NOT use for prediction
- Examples:
  • top merchants
  • transaction counts
  • revenue analysis
  • merchant statistics
""",
        return_direct=True
    )

    prediction_tool = Tool(
        name="Prediction Tool",
        func=prediction_tool_func,
        description="""
Use for fraud and risk prediction.

- Automatically fetches data from database
- No input required from user
- Returns high-risk merchants
"""
    )

    return [sql_tool, prediction_tool]
```